An Empirical Comparison of Genetic Operators for the Traveling Salesperson Problem on TSPLIB Instances

Emre Yıldız, Anıl Aygün, Mustafa Yiğit Güzel, Meriç Özkayagan
Department of Computer Engineering, Ege University, Izmir, Turkey
Student IDs: 05210000222, 05210000229, 05210000209, 05230001155

Abstract—The Traveling Salesperson Problem (TSP) is a canonical NP-hard combinatorial optimization benchmark and one of the most studied test beds for genetic algorithms (GAs). Despite five decades of literature, practitioners must still choose between a large catalogue of selection schemes, recombination operators and mutation strategies whose interactions are problem-specific. In this work, we implement a permutation-encoded GA from scratch in TypeScript and use it to systematically compare three mutation operators (swap, inversion, scramble), two crossover operators (Order Crossover OX1, Partially Mapped Crossover PMX) and three selection schemes (tournament, roulette wheel, linear ranking) on the standard TSPLIB instances berlin52 and eil51. Each configuration is repeated over ten independent random seeds with a population of 200 and 500 generations. The results show that (i) inversion mutation reduces the mean optimality gap on berlin52 from 19.97% (swap) to 9.35%, (ii) tournament selection with $k = 5$ dominates both ranking and roulette wheel by more than 15 percentage points of gap, and (iii) PMX and OX1 are statistically indistinguishable on berlin52. The implementation is delivered as an interactive Next.js web application with a live evolution visualization, a built-in batch experiment runner, and complete reproducibility through seedable pseudo-random number generation.

Index Terms—Genetic algorithm, Traveling Salesperson Problem, TSPLIB, permutation encoding, evolutionary computing, empirical comparison.

I. Introduction

The Traveling Salesperson Problem (TSP) asks for a Hamiltonian cycle of minimum total length over a set of n cities. With a search space of size $(n-1)!/2$ for the symmetric case, exact methods become intractable beyond a few hundred cities, motivating the use of metaheuristics. Among evolutionary techniques, genetic algorithms (GAs) have been applied to the TSP since the late 1980s and remain a strong baseline against which more specialised heuristics are compared.

A practical GA for TSP is parameterised along several orthogonal axes: the representation (path, adjacency, ordinal), the selection operator (proportional, tournament, ranking), the crossover operator (OX1, PMX, ERX, CX, ...), the mutation operator (swap, inversion, insertion, scramble), and a number of scalar hyperparameters such as population size, mutation rate, crossover rate and elitism. Theoretical analyses provide useful guidance, but empirical comparisons remain the primary tool for narrowing this design space on a given instance.

The objective of this project is twofold:

- to provide a self-contained, fully reproducible GA implementation that runs in the browser and visualises evolution generation by generation;
- to deliver a controlled empirical comparison of the most common genetic operators on two well-studied TSPLIB instances.

The remainder of the paper is organised as follows. Section II summarises related work. Section III details the methodology and operators. Section IV reports experimental results, and Section V concludes.

II. Related Work

We surveyed eight studies that span the four decades of GA-for-TSP literature.

1) Goldberg and Lingle (1985) [1] introduced Partially Mapped Crossover (PMX), the first crossover operator specifically designed for permutation chromosomes. PMX preserves a random window from one parent and remaps conflicting genes from the other parent through a position-based bijection. PMX is one of the two crossover operators we benchmark.

2) Davis (1985) [2] proposed Order Crossover (OX1). Unlike PMX, OX1 preserves the relative order of cities in the non-window region rather than absolute positions, which is widely considered a better fit for TSP because tour quality depends on adjacencies rather than absolute indices. OX1 is our second benchmarked crossover.

3) Whitley et al. (1989) [3] studied selection pressure systematically and showed that ranking and tournament schemes dominate proportional (roulette) selection on combinatorial problems because the latter suffers from premature convergence when the fitness distribution becomes flat. Our experiments reproduce this effect in a modern implementation.

4) Larrañaga et al. (1999) [4] provided a comprehensive survey of representations and crossover operators for the TSP, comparing OX1, PMX, CX, ERX and several variants on small TSPLIB instances. Their methodology of averaging over multiple seeds informs our own experimental protocol.

5) Reinelt [5] introduced the TSPLIB benchmark collection, including the berlin52 and eil51 instances we use. Known optima are 7542 and 426 respectively. Reporting the optimality gap—the relative excess over the known optimum—is the de facto standard in the literature.

6) Potvin (1996) [6] reviewed GAs for the TSP and discussed mutation operators, arguing that 2-opt-style segment-reversing operators (inversion) typically outperform swap mutation because they preserve more existing high-quality edges. Our results corroborate this finding.

7) Nagata and Kobayashi (2013) [7] introduced Edge Assembly Crossover (EAX), a state-of-the-art TSP-specific operator that reaches near-optimal solutions on instances with thousands of cities. EAX is outside the scope of our comparison—which focuses on classical operators—but it sets a useful upper bound for the achievable solution quality on benchmark instances.

8) Eiben and Smith (2015) [8] provided the textbook treatment of evolutionary computing that we used as the conceptual reference for our taxonomy of operators and for the recommended default hyperparameters that anchor our parameter sweeps.

Across these works, two recurring observations stand out: tournament selection is robust across problem domains, and operator choice matters more than fine-grained tuning of mutation/crossover rates. Our experimental section quantifies both observations on a modern TypeScript implementation.

III. Methodology

A. Representation and Fitness

A candidate solution is a permutation $\pi = (\pi_0, \pi_1, \ldots, \pi_{n-1})$ of city indices, interpreted as a closed tour $\pi_0 \to \pi_1 \to \cdots \to \pi_{n-1} \to \pi_0$. The fitness function to be minimised is the Euclidean tour length

$$L(\pi) = \sum_{i=0}^{n-1} d(c_{\pi_i}, c_{\pi_{(i+1) \bmod n}}),$$

where c_i are the city coordinates and d is the Euclidean distance. Distances are pre-computed once into an $n \times n$ Float64Array to avoid repeated $\sqrt{\cdot}$ evaluations during fitness queries.

B. Initialisation

The initial population of μ individuals is sampled uniformly at random from the space of permutations using a Fisher–Yates shuffle. The shuffle—and every subsequent stochastic decision—is driven by a seedable Mulberry32 pseudo-random number generator, which makes every experiment exactly reproducible from a single integer seed.

C. Selection

We compare three selection schemes:

- Tournament: k individuals are sampled uniformly with replacement and the fittest is chosen. We treat k as a hyperparameter and report results for $k = 2$ and $k = 5$.
- Fitness-proportional (roulette): weights are $1/(L(\pi) + \varepsilon)$ to convert minimisation to maximisation; an individual is sampled with probability proportional to its weight.
- Linear ranking: individuals are sorted by fitness and sampled with probability proportional to their rank. Rank 1 (best) receives weight μ, rank μ (worst) receives weight 1.

D. Crossover

Order Crossover (OX1). Given two parents p_1, p_2, a random window $[\ell, h]$ is copied from p_1 into the child. The remaining positions are filled with the cities of p_2 in their relative order, starting from $h+1$ and skipping cities already present.

Partially Mapped Crossover (PMX). The window from p_1 is copied into the child; the remaining positions are inherited from p_2 but any gene that conflicts with the window is repeatedly remapped through the position-wise bijection $p_1[i] \leftrightarrow p_2[i]$ defined by the window.

Crossover is applied with probability p_c; otherwise the child is a clone of p_1.

E. Mutation

Swap. Two positions are exchanged.

Inversion. The segment between two random positions is reversed, which is equivalent to a 2-opt move performed on the chromosome.

Scramble. The segment between two random positions is shuffled uniformly at random.

Mutation is applied with probability p_m per offspring.

F. Replacement and Elitism

We use generational replacement with elitism: the top E individuals are copied unchanged into the next generation, and the remaining $\mu - E$ slots are filled by selection + crossover + mutation.

G. Termination

The algorithm terminates after a fixed budget of G generations. We do not use a stagnation-based stopping criterion because the goal is to compare configurations under an identical computational budget.

H. Implementation

The GA, the TSPLIB data and a live visualisation are implemented in TypeScript and served as a Next.js 16 application. Distance matrices are stored as flat Float64Arrays. The browser application can step generation-by-generation under requestAnimationFrame, while a parallel headless runner (tsx scripts/run-experiments.ts) produces all experimental tables in this paper. The full source is included in the project archive.

I. Default Configuration

Unless otherwise stated, parameter sweeps fix the following base configuration: $\mu = 200$, $G = 500$, $p_c = 0.9$, $p_m = 0.2$, $E = 4$, selection = tournament with $k = 5$, crossover = OX1, mutation = inversion.

IV. Experimental Work

A. Setup

We evaluate on two TSPLIB instances: berlin52 (52 cities, optimum 7542) and eil51 (51 cities, optimum 426). Each configuration is run for ten independent seeds $(1, 2, \ldots, 10)$; we report the mean and standard deviation of the best tour length over those ten runs, together with the mean optimality gap

$$\text{gap}(\%) = \frac{1}{R} \sum_{r=1}^{R} \frac{L_{\text{best}}^{(r)} - L^*}{L^*} \cdot 100.$$

B. Mutation Operator

Table I compares swap, inversion and scramble mutation under otherwise identical settings.

TABLE I
Mutation operator comparison (mean ± std over 10 seeds).

Mutation	berlin52		eil51	
	Mean best	Gap (%)	Mean best	Gap (%)
swap	9048.44	19.97	505.84	18.74
inversion	8247.00	9.35	456.40	7.14
scramble	9201.33	22.00	537.06	26.07

Inversion mutation is uniformly the strongest operator, halving the mean gap relative to swap on berlin52 and reducing it by a factor of 2.6 on eil51. The reason is well documented in the literature [6]: a single inversion is exactly a 2-opt move, which preserves $n - 3$ existing edges out of n. Swap mutation, by contrast, always destroys four edges, and scramble destroys an even larger neighbourhood, both of which act as a near-uniform restart on already-good tours. These results corroborate Potvin's observation quantitatively in our setting.

C. Crossover Operator

Table II compares OX1 and PMX.

TABLE II
Crossover operator comparison (mean ± std over 10 seeds).

Crossover	berlin52		eil51	
	Mean best	Gap (%)	Mean best	Gap (%)
OX1	8247.00	9.35	456.40	7.14
PMX	8159.49	8.19	457.67	7.43

OX1 and PMX are statistically indistinguishable: the difference of 87 units on berlin52 is well below either standard deviation ($\sigma = 335$ for OX1, $\sigma = 156$ for PMX), and the difference reverses sign on eil51. This matches Larrañaga et al.'s finding [4] that no single classical crossover dominates on small TSPLIB instances; operator-level differences become apparent only at larger problem sizes or under stricter computational budgets.

D. Selection Method

Table III compares the three selection schemes.

TABLE III
Selection method comparison (mean ± std over 10 seeds).

Selection	berlin52		eil51	
	Mean best	Gap (%)	Mean best	Gap (%)
tournament $k = 5$	8247.00	9.35	456.40	7.14
tournament $k = 2$	9452.48	25.33	550.11	29.13
roulette	13213.97	75.21	752.04	76.53
rank	9490.79	25.84	542.76	27.41

This is the largest effect we measured. Roulette wheel selection catastrophically fails on both instances, leaving the algorithm essentially no better than random search after 500 generations. The cause is well-known and was already analysed by Whitley [3]: when fitness values cluster within a narrow range, all individuals receive nearly equal selection probabilities, which collapses the selection pressure and leaves the GA driven only by mutation. Rank selection partly mitigates this but cannot match the deterministic, scale-invariant pressure of tournament selection. Increasing the tournament size from 2 to 5 also halves the gap, confirming that selection pressure—not selection type—is the dominant lever.

E. Mutation Rate

TABLE IV
Mutation rate sweep on berlin52.

p_m	Mean best	Std	Gap (%)
0.05	8458.18	209.34	12.15
0.10	8224.06	337.43	9.04
0.20	8247.00	335.00	9.35
0.40	8224.63	174.88	9.05
0.80	8197.03	208.74	8.69

The GA is remarkably robust to the mutation rate in the range $[0.10, 0.80]$: the mean best length varies by less than 0.8%. Only the very small $p_m = 0.05$ setting visibly underperforms, presumably because inversion-driven exploration is then too rare to escape local optima. This robustness is consistent with Eiben and Smith's recommendation [8] that mutation rates can usually be left at their defaults provided the operator is well chosen.

F. Population Size

Increasing μ from 50 to 400 reduces the gap on berlin52 from 11.72% to 6.68% and also reduces variance, but at a roughly linear increase in wall-clock cost per generation. The marginal returns flatten beyond $\mu = 400$, suggesting

TABLE V
Population size sweep on berlin52.

μ	Mean best	Std	Gap (%)
50	8425.90	273.68	11.72
100	8275.63	366.35	9.73
200	8247.00	335.00	9.35
400	8045.51	198.81	6.68

that the budget is better spent on post-processing (e.g. a 2-opt local search on the elite, an idea we leave for future work).

G. Best Configuration

The best configuration we identified, defined as the parameter combination yielding the lowest mean optimality gap, is: tournament selection with $k = 5$, OX1 crossover, inversion mutation, $p_m = 0.4$, $\mu = 400$, $E = 10$, $G = 500$. With this configuration the mean tour length on berlin52 is ∼8000 (gap ∼6%) and on eil51 ∼450 (gap ∼5%). State-of-the-art TSP-specific operators such as EAX [7] reach gaps below 1%, but rely on instance-specific edge-assembly information that is outside the scope of a generic GA.

V. Conclusion and Discussion

We presented a from-scratch TypeScript implementation of a permutation-encoded genetic algorithm for the TSP, together with an interactive web-based visualiser and a headless batch experiment runner. Across two TSPLIB instances, ten random seeds and six controlled parameter sweeps, the most significant findings are:

1) Operator choice dominates rate tuning. Switching the selection scheme or mutation operator moves the mean gap by 10–65 percentage points, while sweeping the mutation rate over an order of magnitude moves it by less than one.
2) Inversion mutation is the right default for TSP. Its 2-opt-equivalent semantics preserve far more good edges than swap or scramble, and the effect is stable across both instances.
3) Tournament selection is essential. Roulette wheel selection fails to produce any meaningful improvement over random search on these instances; this is a textbook effect that we reproduce quantitatively.
4) Population size has diminishing returns. Beyond $\mu = 400$ the budget is more profitably spent on local search.

Future work could extend the implementation along three directions: (i) replace inversion mutation with full 2-opt local search to obtain a memetic algorithm, which on TSPLIB benchmarks is known to bring the gap below 1%; (ii) compare against state-of-the-art TSP operators such as Edge Assembly Crossover [7]; and (iii) port the algorithm into a Web Worker to remove the main-thread bottleneck and enable larger experiments directly inside the visualisation.

Member Contributions

Emre Yıldız (05210000222) — Problem framing and literature review (sourced the eight cited studies, synthesised the methodological taxonomy); primary author of the Abstract, Introduction and Related Work sections.

Anıl Aygün (05210000229) — GA core implementation (permutation representation, OX1/PMX crossover, tournament/roulette/rank selection, elitism, generational replacement); primary author of the Methodology section.

Mustafa Yiğit Güzel (05210000209) — Mutation operators, TSPLIB integration, headless experiment runner, parameter sweeps across ten random seeds; primary author of the Experimental Work section.

Meriç Özkayagan (05230001155) — Interactive web visualiser (live route animation, fitness chart, control panel), end-to-end testing, Conclusion and Future Work sections, demo script.

AI Usage Statement

ChatGPT (GPT-4 class) and GitHub Copilot were used during development. Specifically, ChatGPT was consulted for clarifying the precise update mechanics of OX1 and PMX, and for outlining the structure of this manuscript. Copilot provided in-editor completions for boilerplate TypeScript code (chart axes, form bindings). All AI-generated outputs were reviewed by the group: code was validated by unit tests against hand-computed examples on small permutations, the OX1/PMX mechanics were cross-checked against Goldberg and Lingle [1] and Davis [2], and any text drafts were rewritten in our own voice and verified against the cited sources.

Data Availability

Source code, raw experimental CSVs, the compiled report and the live visualisation are available in the project repository at https://github.com/mericozkayagan/evocomp. The TSPLIB instances berlin52 and eil51 are part of the public TSPLIB95 archive (http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/).

References

[1] D. E. Goldberg and R. Lingle, "Alleles, loci, and the traveling salesman problem," in Proceedings of the 1st International Conference on Genetic Algorithms, 1985, pp. 154–159.

[2] L. Davis, "Applying adaptive algorithms to epistatic domains," in Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI), 1985, pp. 162–164.

[3] L. D. Whitley, "The GENITOR algorithm and selection pressure: Why rank-based allocation of reproductive trials is best," in Proceedings of the Third International Conference on Genetic Algorithms, 1989, pp. 116–121.

[4] P. Larrañaga, C. M. H. Kuijpers, R. H. Murga, I. Inza, and S. Dizdarevic, "Genetic algorithms for the travelling salesman problem: A review of representations and operators," Artificial Intelligence Review, vol. 13, no. 2, pp. 129–170, 1999.

[5] G. Reinelt, "TSPLIB—A traveling salesman problem library," ORSA Journal on Computing, vol. 3, no. 4, pp. 376–384, 1991.

[6] J.-Y. Potvin, "Genetic algorithms for the traveling salesman problem," Annals of Operations Research, vol. 63, no. 3, pp. 337–370, 1996.

[7] Y. Nagata and S. Kobayashi, "A powerful genetic algorithm using edge assembly crossover for the traveling salesman problem," INFORMS Journal on Computing, vol. 25, no. 2, pp. 346–363, 2013.
[8] A. E. Eiben and J. E. Smith, Introduction to Evolutionary Computing, 2nd ed. Berlin: Springer, 2015.